Sistemi Operativi e Reti di Calcolatori (SOReCa)

Corso di Laurea in *Ingegneria Informatica e Automatica (BIAR)*
Terzo Anno | Primo Semestre
A.A. 2025/2026

Esercitazione [07] Client/Server con Socket

Riccardo Lazzeretti lazzeretti@diag.uniroma1.it
Paolo Ottolino paolo.ottolino@uniroma1.it
Matteo Cornacchia cornacchia@diag.uniroma1.it

DIPARTIMENTO DI INGEGNERIA INFORMATICA AUTOMATICA E GESTIONALE ANTONIO RUBERTI

SAPIENZA UNIVERSITÀ DI ROMA

Sommario

- Soluzioni precedente esercitazione
- TCP/IP
  - Byte Order: network vs host
- TCP:
  - Client/Server
  - Protocollo: messaggi testuali
- Esercizio
  - EchoServer
- UDP
- Esercizio
  - EchoServer

Dipartimento di Ingegneria informatica
automatica e gestionale Antonio Ruberti

Sapienza
Università di Roma

Soluzione: FIFO Produttore-Consumatore

- Lab06, Esercizio 1

Dipartimento di Ingegneria informatica
automatica e gestionale Antonio Ruberti



Sapienza
Università di Roma

Produttore/Consumatore

Lab06 es1: Producer-Consumer con FIFO

- L'applicazione è sviluppata in due moduli separati.
- Si tiene conto della configurazione con `NUM_CONSUMERS` consumatori e `NUM_PRODUCERS` produttori
- Non serve realmente un buffer, ma si usa una FIFO

- Completare il codice dell'applicazione produttore/consumatore
- Sorgenti
  - `makefile`
  - `producer.c`
  - `consumer.c`
- Suggerimento: seguire i blocchi di commenti inseriti nel codice
- Suggerimento: il producer sarà il primo a operare e il consumer l'ultimo
  - Quindi il producer crea la FIFO e il consumer la distrugge
  - La FIFO ha una grande dimensione e si satura difficilemente
    - Ridimensionare la fifo
      - #define _GNU_SOURCE //da mettere a inizio del file producer.c, prima delle include
      - fcntl(fifo, F_SETPIPE_SZ, 10*sizeof(int)); //posizionare subito dopo la open
    - Rallentare il consumer con delle sleep()
- Test:
  - Lanciate prima `producer` (crea semafori e memoria condivisa) e poi `consumer`

Shared Memory

Lab06-es1: Producer con FIFO

```
void initFIFO() {
    /** COMPLETE THE FOLLOWING CODE BLOCK
     *
     * Request the kernel to create a FIFO and open it
     **/
    // Create the FIFO
    int ret = mkfifo(FIFO_NAME, 0666);
    if(ret) handle_error("Cannot create FIFO");

    fifo = open(FIFO_NAME, O_WRONLY);
    if(fifo == -1) handle_error("Cannot open FIFO for writing");
    fcntl(fifo, F_SETPIPE_SZ, 10*sizeof(int));
}
```

```
static void closeFIFO() {

    /** [TO DO] Method to close and remove the FIFO
     *
     * - Close the fifo
     * - Destroy the fifo
     * */

    // close the descriptors
    int ret = close(fifo);
    if(ret) handle_error("Cannot close FIFO");
}
```

```
int writeValue(int value) {

    int ret;
    int bytes_sent = 0;
    while (bytes_sent != sizeof(int)) {
        ret = write(fifo, &value, sizeof(int));
        if (ret == -1 && errno == EINTR) continue;
        if (ret == -1) handle_error("Cannot write to FIFO");
        if (ret != sizeof(int)) handle_error_en(ret, "Partial write to
FIFO");
        bytes_sent = ret;
    }
    return bytes_sent;
}
```

```
void produce(int id, int numOps) {
    int localSum = 0;
    while (numOps > 0) {
        // producer, just do your thing!
        int value = performRandomTransaction();

        writeValue(value); // write value in the buffer inside the shared memory
        /**/

        localSum += value;
        numOps--; // update the producer position
    }
    printf("Producer %d ended. Local sum is %d\n", id, localSum);
}
```

DIPARTIMENTO DI INGEGNERIA INFORMATICA
AUTOMATICA E GESTIONALE ANTONIO RUBERTI

SAPIENZA
UNIVERSITÀ DI ROMA

Shared Memory

Lab06-es1: Consumer con FIFO

```
void openFIFO() {
    /** COMPLETE THE FOLLOWING CODE BLOCK
     *
     * Request the kernel to create a FIFO and open it
     **/

    fifo = open(FIFO_NAME, O_RDONLY);
    if(fifo == -1) handle_error("Cannot open FIFO for writing");

}
```

```
static void closeFIFO() {

    // close the descriptors
    int ret = close(fifo);
    if(ret) handle_error("Cannot close FIFO");

    // destroy the FIFO
    ret = unlink(FIFO_NAME);
    if(ret) handle_error("Cannot unlink FIFO");
}
```

```
int readValue(int * value) {

    int ret;

int bytes_read = 0;
    do {
        ret = read(fifo, value, sizeof(int));
        if (ret == -1 && errno == EINTR) continue;
        if (ret == -1) handle_error("Cannot read from FIFO");
        if (ret ==  0) handle_error("Process has closed the FIFO
unexpectedly! Exiting...\n");
        if (ret < sizeof(int)) handle_error_en(ret, "partial read from
FIFO");
        // we use post-increment on bytes_read so that we first read the
        // byte that has just been written, then we do the increment
        bytes_read += ret;
    } while(bytes_read != sizeof(int));
    return bytes_read;
}
```

```
void consume(int id, int numOps) {
    int localSum = 0;
    int value;
    while (numOps > 0) {

        /**
         * Complete the following code:
         * write value in the buffer inside the shared memory and update the producer
position
         */
        readValue(&value);

        localSum += value;
        numOps--;
    }
    printf("Consumer %d ended. Local sum is %d\n", id, localSum);
}
```

Dipartimento di Ingegneria informatica
automatica e gestionale Antonio Ruberti

Sapienza
Università di Roma

Soluzione: Time Server

- Lab06, Esercizio 2

Dipartimento di Ingegneria informatica
automatica e gestionale Antonio Ruberti

Sapienza
Università di Roma

Invio e Ricezione messaggi su Socket

Lab06 es2: TimeServer

- Scenario
  - Due processi: un client ed un server
  - Il server è in ascolto in attesa di connessioni TCP
  - Il client si connette al server ed invia un comando
    - Messaggio «TIME»
  - Se il server riceve il messaggio atteso, la risposta conterrà ora e data correnti, altrimenti manderà un messaggio di errore
- Sorgenti: `server.c` e `client.c`
- Esercizio
  - Completare le parti mancanti, relative all'invio/ricezione di messaggi via socket
  - Per l'esecuzione, è necessario lanciare client e server su terminali diversi



Dipartimento di Ingegneria informatica
automatica e gestionale Antonio Ruberti

Sapienza
Università di Roma

Shared Memory

Lab06 es2: TimeServer - Server

```
int main(int argc, char* argv[]) {
int main(int argc, char* argv[]) {
    int ret;

    int socket_desc, client_desc;
    /********* socket() bind() listen() ******************/
    // loop to handle incoming connections serially
    while (1) {
        client_desc = accept(socket_desc, (struct sockaddr*) &client_addr, (socklen_t*) &sockaddr_len); // BLOCCANTE
        if (client_desc < 0) handle_error("Cannot open socket for incoming connection");
        if (DEBUG) fprintf(stderr, "Incoming connection accepted...\n");
        connection_handler(client_desc);

        if (DEBUG) fprintf(stderr, "Done!\n");
    }

    exit(EXIT_SUCCESS); // this will never be executed
}
```

```
void connection_handler(int socket_desc) {
    /* … */
    // receive command from client
    char recv_buf[256];
    size_t recv_buf_len = sizeof(recv_buf);
    int recv_bytes;
    recv_bytes = 0;
    do { //prima ricevo il byte…
        ret = recv(socket_desc, recv_buf + recv_bytes, recv_buf_len-recv_bytes, 0);
        if (ret == -1 && errno == EINTR) continue; // system call interrotta da segnale □ ignora errore e continua.
        if (ret == -1) handle_error("Cannot read from the socket"); //errore di lettura reale □ terminazione del programma.
        if (ret == 0) break; // l'altro peer ha chiuso la comunicazione □ uscita dal ciclo.
        recv_bytes += ret;
        if (DEBUG) fprintf(stderr, "Message of %d bytes received\n", recv_bytes);
    } while ( recv_buf[recv_bytes-1] != '\n' ); // poi controllo che byte ho ricevuto (in C si conta da 0)
 /* … */
```

```
void connection_handler(int socket_desc) {
/* … */
/** INSERT CODE TO SEND DATA HERE
int bytes_sent=0;
    while ( bytes_sent < server_message_len-1) {
        ret = send(socket_desc, send_buf + bytes_sent, server_message_len -1 - bytes_sent, 0);
        if (ret == -1 && errno == EINTR) continue; // system call interrotta da segnale □ ignora errore e continua.
        if (ret == -1) handle_error("Cannot write to the socket"); //errore di lettura reale □ terminazione del programma.
        // send() non restituisce mai il valore 0, quindi non c'e' if (ret == 0)
        bytes_sent += ret;
    }
```

DIPARTIMENTO DI INGEGNERIA INFORMATICA
AUTOMATICA E GESTIONALE ANTONIO RUBERTI

SAPIENZA
UNIVERSITÀ DI ROMA

Shared Memory

Lab06 es2: TimeServer - Client

```
int main(int argc, char* argv[]) {
    int ret;

    // variables for handling a socket
    int socket_desc;
    struct sockaddr_in server_addr = {0}; // some fields are required to be filled with 0
    // create a socket
    socket_desc = socket(AF_INET, SOCK_STREAM, 0);
    if (socket_desc < 0) handle_error("Could not create socket");
    /* … */
}
```

```
// send command to server
    char* command = SERVER_COMMAND;
    size_t command_len = strlen(command);

    /** [SOLUTION]
     *
     * Suggestions:
     * - send() with flags = 0 is equivalent to write() on a descriptor
     *
     * For the time being we won't deal with partially sent messages.
     */
    int bytes_sent=0;
    while ( bytes_sent < command_len) {
        ret = send(socket_desc, command + bytes_sent, command_len - bytes_sent, 0);
        if (ret == -1 && errno == EINTR) continue; // system call interrotta da segnale □ ignora errore e continua.
        if (ret == -1) handle_error("Cannot write to the socket"); //errore di lettura reale □ terminazione del programma.
        // send() non restituisce mai il valore 0, quindi non c'e' if (ret == 0)
        bytes_sent += ret;
    }
```

```
// read message from the server
    char recv_buf[256];
    size_t recv_buf_len = sizeof(recv_buf);
    int recv_bytes;
    recv_bytes = 0;
    do {
        ret = recv(socket_desc, recv_buf + recv_bytes, recv_buf_len-recv_bytes, 0);
        if (ret == -1 && errno == EINTR) continue; // system call interrotta da segnale □ ignora errore e continua.
        if (ret == -1) handle_error("Cannot write to the socket"); //errore di lettura reale □ terminazione del programma.
        if (ret == 0) break; // l'altro peer ha chiuso la comunicazione □ uscita dal ciclo.
        recv_bytes += ret;
    } while ( recv_buf[recv_bytes-1] != '\n' );
   /* … */
```

DIPARTIMENTO DI INGEGNERIA INFORMATICA
AUTOMATICA E GESTIONALE ANTONIO RUBERTI

SAPIENZA
UNIVERSITÀ DI ROMA

TCP/IP: Funzioni Primitive

Byte Order: Network, Host

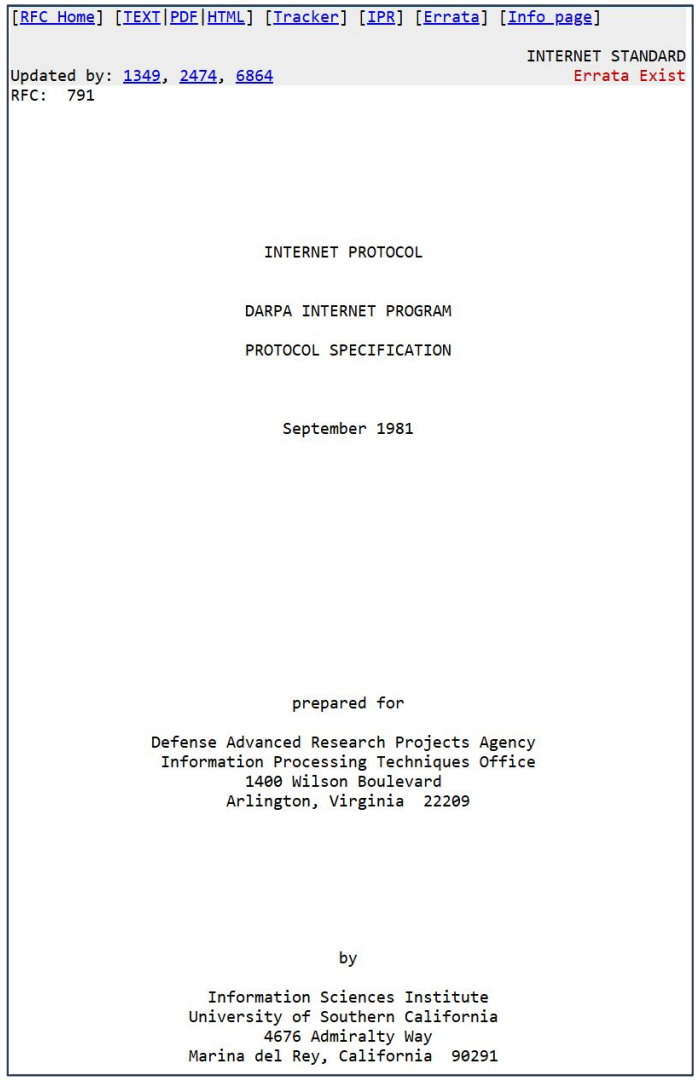
Dipartimento di Ingegneria informatica
automatica e gestionale Antonio Ruberti



Sapienza
Università di Roma

Obiettivi

1. Capire la differenza tra network byte order e host byte order (RFC 791: https://www.rfc-editor.org/rfc/rfc791)



[RFC Home] [TEXT|PDF|HTML] [Tracker] [IPR] [Errata] [Info page]

INTERNET STANDARD
Updated by: 1349, 2474, 6864 Errata Exist
RFC: 791

INTERNET PROTOCOL

DARPA INTERNET PROGRAM

PROTOCOL SPECIFICATION

September 1981

prepared for

Defense Advanced Research Projects Agency
Information Processing Techniques Office
1400 Wilson Boulevard
Arlington, Virginia 22209

by

Information Sciences Institute
University of Southern California
4676 Admiralty Way
Marina del Rey, California 90291

Big Endian era il formato predominante (come per IBM System/360) ai tempi del design originale di ARPANet.

Funzioni Primitive

Byte Order: network e host

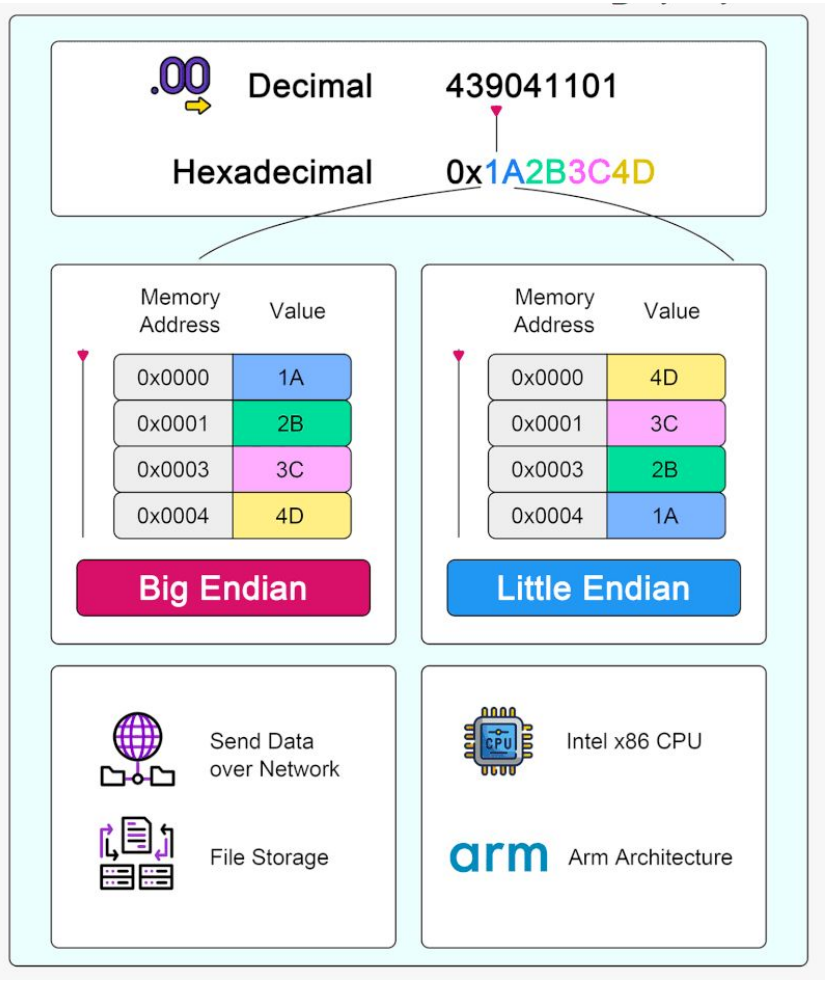
Nello scambio di dati numerici tra macchine con architetture (potenzialmente) differenti, occorre verificare il **byte order**

- Un dato numerico è rappresentato come una sequenza di byte
- *Il primo byte di tale sequenza è il più significativo* (**Big Endian**) *o il meno significativo* (**Little Endian**)*?*

- Dati numerici scambiati tra macchine che usano byte order diversi (**host byte order**) vengono interpretati in maniera diversa

- La maggior parte dei protocolli di rete (inclusi IPv4 e TCP) usano Big Endian come **network byte order** nell'header

- La maggior parte delle architetture CPU (Intel, AMD, ARM) sono Little Endian come **host byte order**



Dipartimento di Ingegneria informatica automatica e gestionale Antonio Ruberti

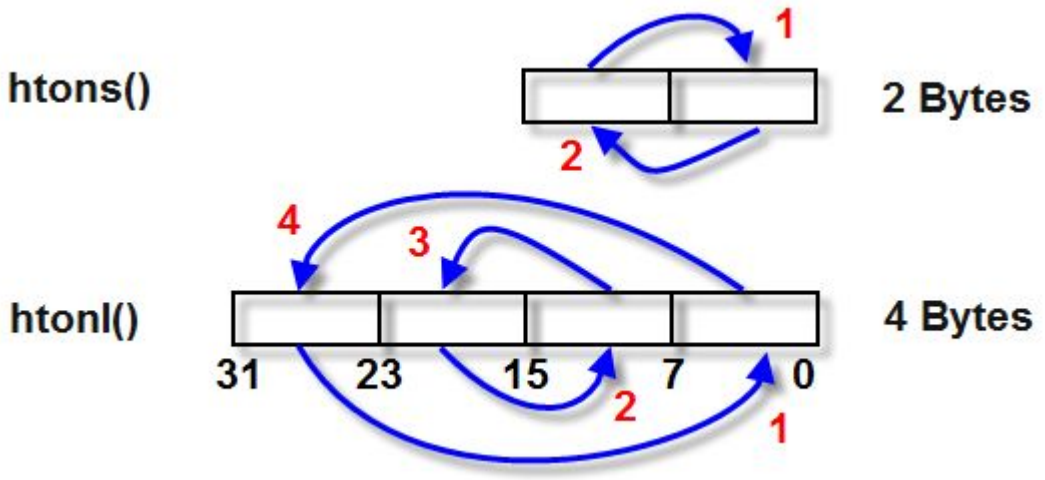
Sapienza Università di Roma

Funzioni Primitive

Byte Order: Funzioni di conversion per la porta

Il numero di porta di una socket TCP può variare tra 0 e 65535

- la definizione del tipo generico `uint16_t` può essere inclusa tramite **`<arpa/inet.h>`** o più in generale `<stdint.h>`
- su Linux IA32 e x86_64 equivale ad un `unsigned short`
- le porte nel range 0-1023 richiedono privilegi di root



- Funzione `htons()` (host-to-network-ushort)
  - `uint16_t htons(uint16_t hostshort);`
  - Converte un ushort da host byte order a network byte order

- Funzione `ntohs()` (network-to-host-ushort)
  - `uint16_t ntohs(uint16_t netshort);`
  - Converte un ushort da network byte order a host byte order

Funzioni Primitive

Byte Order: Funzioni di conversion per l'Indirizzo

Un indirizzo IPv4 è rappresentato con `struct in_addr`
Il campo `sin_addr` di `struct sockaddr_in` è infatti di tipo `struct in_addr`

- *Reminder*: variabili di tipo `struct sockaddr_in` vengono usate nella `bind()` e nella `accept()` lato server e nella `connect()` lato client

`struct in_addr` contiene il campo `s_addr` di tipo `in_addr_t` che rappresenta l'indirizzo in network byte order
Entrambe le strutture sono definite in `<netinet/in.h>`

```
in_addr_t inet_addr(const char *cp)
```

- Converte un indirizzo IPv4 dalla forma dotted string (x.y.z.w) al network byte order
- Il valore di ritorno viene di solito assegnato al campo `s_addr` di `struct in_addr`

```
const char *inet_ntop(int af, const void *src, char *dst, socklen_t n);
```

- Converte l'indirizzo di rete `src` della address family `af` in una stringa di lunghezza `n` e la copia in `dst`
- Ritorna un puntatore a `dst`, oppure `NULL` in caso di errore
- Le macro `AF_INET` e `INET_ADDRSTRLEN` possono essere usate rispettivamente per il primo e l'ultimo argomento
  - Quanto vale `INET_ADDRSTRLEN`?

TCP: Funzioni Primitive

Client/Server: Socket
Protocollo: messaggi

Dipartimento di Ingegneria informatica
automatica e gestionale Antonio Ruberti



Sapienza
Università di Roma

Obiettivi

1. Imparare ad impostare un'applicazione client/server che preveda:
   - Server single-thread
     - Come mettersi in ascolto su una porta nota?
     - Come accettare una connessione da client?
   - Client
     - Come connettersi ad un server in ascolto?

2. Semplice protocollo basato su messaggi testuali

Dipartimento di Ingegneria informatica
automatica e gestionale Antonio Ruberti

Sapienza
Università di Roma

Funzioni Primitive TCP

Socket (Server, Client)

Richiedere al kernel di poter comunicare

- `socket()` creare i descrittori di comunicazione (struttura dati del kernel) per un **protocollo** supportato (`/etc/protocols`)

Mettersi in ascolto su una porta nota

- `bind()` collegare socket ed indirizzo locale: al processo arriveranno tutte le comunicazioni sulla **porta dichiarata** (`/etc/services`)
- `listen()` dichiarare il **back-log** (numero max di connessioni accettabili) (*)
- `accept()` avviare l'attesa delle richieste di comunicazione

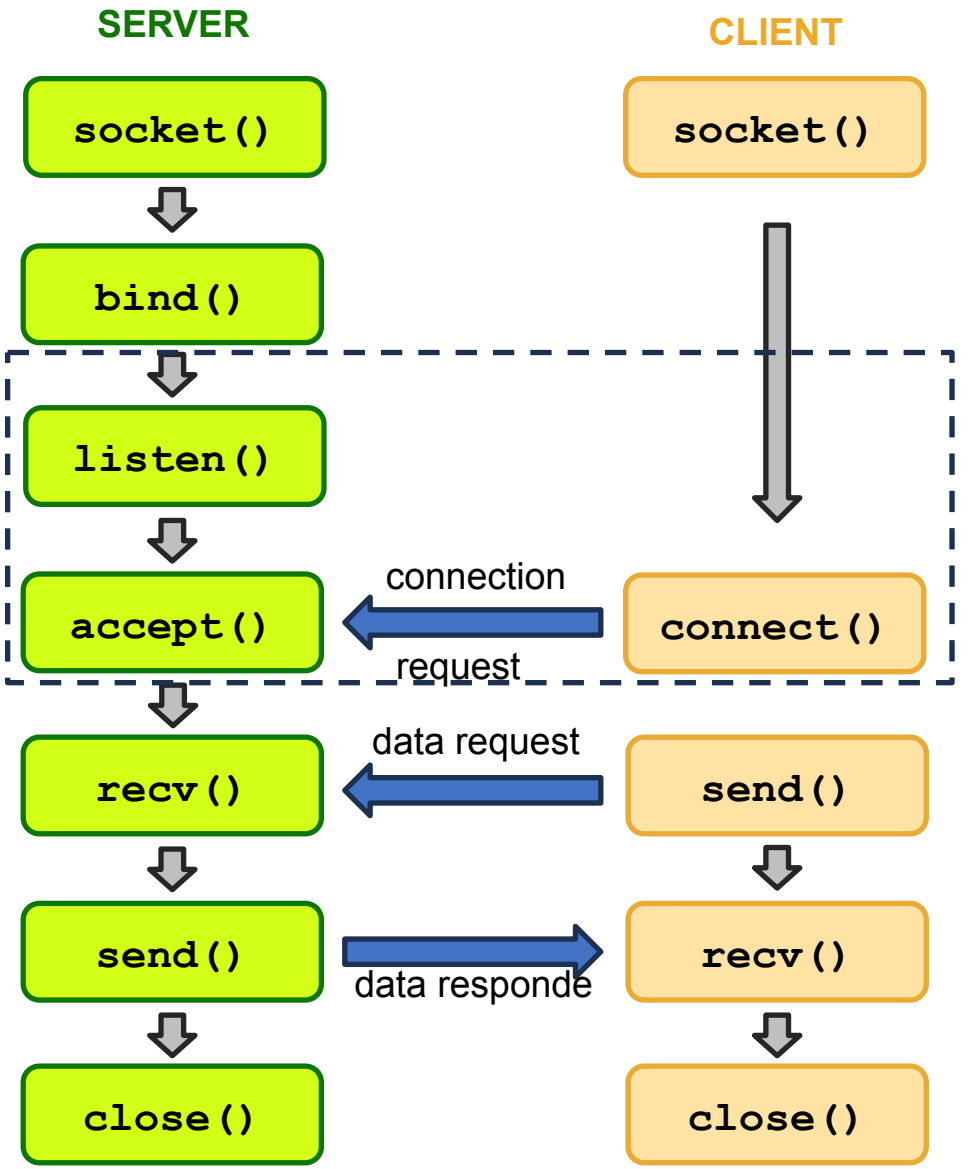
Inviare una richiesta di comunicazione

- `connect()` inviare una richiesta di comunicazione ad un **indirizzo:porta**

Scambiarsi le informazioni tra server e client

- `send()` mandare messaggi sulla socket (indirizzo:porta)
- `recv()` ricevere messaggi sulla socket (indirizzo:porta)
- `close()` liberare le strutture dati del kernel (una volta terminato lo scambio di messaggi)

(*) SYN – SYN/ACK – ACK SYN Flooding



DIPARTIMENTO DI INGEGNERIA INFORMATICA AUTOMATICA E GESTIONALE ANTONIO RUBERTI

SAPIENZA UNIVERSITÀ DI ROMA

Funzioni Primitive TCP

Socket (Server, Client): Strutture Dati

`struct in_addr`: rappresenta un indirizzo IP a 32 bit

`struct sockaddr_in`: descrizione di una socket; al suo interno le informazioni principali sono:

- Famiglia dell'indirizzo (`sin_family`)
  - Per i nostri scopi, `AF_INET`: protocollo IPv4
  - Ne esistono altre, es: `AF_UNIX`, `AF_BLUETOOTH`
- Indirizzo IP (`sin_addr.s_addr`), per i nostri scopi:
  - Lato server, `INADDR_ANY`: in ascolto su tutte le interfacce
  - Lato client, specifica l'indirizzo IP del server
- Numero porta (`sin_port`)
  - Bisogna rispettare l'ordine di trasmissione dei byte per la rete
  - `sin_port = htons(port)` per invertire l'ordine dei bytes

Funzioni Primitive TCP

Socket (Server, Client): funzione `socket()`

```
int socket(int family, int type, int protocol);
```

Crea una socket, ossia un endpoint di comunicazione

Argomenti

- `family`: per i nostri scopi, `AF_INET`
  (vedi struttura dati `struct sockaddr_in`)
- `type`: per i nostri scopi, `SOCK_STREAM` (protocollo TCP)
  - Ne esistono altre, es: `SOCK_DGRAM` (protocollo UDP)
- `protocol`: per i nostri scopi, `0`

Valore di ritorno

- In caso di successo, il descrittore della socket
- In caso di errore, `-1`, `errno` è settato

Dipartimento di Ingegneria informatica
automatica e gestionale Antonio Ruberti

Sapienza
Università di Roma

Funzioni Primitive TCP

Socket (Server): funzione `bind()`

```
int bind(int fd, const struct sockaddr *addr, socklen_t len);
```

Assegna un indirizzo ad una socket

Argomenti

- `fd`: descrittore della socket (restituito da `socket()`)
- `addr`: puntatore ad una struttura dati che specifica l'indirizzo
  - Per i nostri scopi: per usare i valori della struttura `struct sockaddr_in` va effettuato il cast a `struct sockaddr`
- `len`: dimensione della struttura dati puntata da `addr`

Valore di ritorno

- In caso di successo, `0`
- In caso di errore, `-1` , `errno` è settato

Funzioni Primitive TCP

Socket (Server): funzione `listen()`

```
int listen(int sockfd, int backlog);
```

Marca la socket come passiva, i.e., specifica che può essere usata per accettare connessioni tramite la funzione `accept()`

Argomenti

- `sockfd`: descrittore della socket (restituito da `socket()`)
- `backlog`: lunghezza massima della coda per le connessioni
  - Se una connessione arriva quando la coda è piena, la connessione viene rifiutata

Valore di ritorno

- In caso di successo, `0`
- In caso di errore, `-1` , `errno` è settato

Funzioni Primitive TCP

Socket (Server): funzione `accept()`

```
int accept(int fd, struct sockaddr *addr, socklen_t *len);
```

Accetta una connessione su una socket in ascolto

- È una chiamata bloccante: rimane in attesa di connessioni

Argomenti

- `fd`: descrittore della socket (restituito da `socket()`)
- `addr`: puntatore ad una struttura dati `struct sockaddr` che verrà riempita con le info della socket del client
- `len`: puntatore ad un intero che verrà settato con la dimensione della struttura dati `addr`

Valore di ritorno

- In caso di successo, un descrittore per comunicare col client
- In caso di errore, `-1` , `errno` è settato

Funzioni Primitive TCP

Socket (Server, Client): funzione `close()`

```
int close(int fd);
```

Nel caso `fd` sia un descrittore di socket, chiude la socket stessa

- `read()` *successive dall'altro endpoint restituiranno* `0` *!!!*

Argomenti

- `fd`: descrittore della socket (ritornato da `socket()`)

Valore di ritorno

- In caso di successo, `0`
- In caso di errore, `-1` , `errno` è settato

Dipartimento di Ingegneria informatica
automatica e gestionale Antonio Ruberti

Sapienza
Università di Roma

Funzioni Primitive TCP

Socket (Client)

Come connettersi ad un server in ascolto?

- Creazione socket - funzione `socket()`
- Connessione al server - funzione `connect()`
- Una volta terminato lo scambio di messaggi, la connessione col client va chiusa - funzione `close()`

Dipartimento di Ingegneria informatica
automatica e gestionale Antonio Ruberti

Sapienza
Università di Roma

Funzioni Primitive TCP

Socket (Client): funzione `connect()`

```
int connect(int fd, const struct sockaddr *addr, socklen_t l);
```

Tenta una connessione su una socket in ascolto

Argomenti

- `fd`: descrittore della socket (ritornato da `socket()`)
- `addr`: puntatore ad una struttura dati `struct sockaddr` che descrive la socket alla quale connettersi (quella del server)
- `l`: dimensione della struttura dati puntata da `addr`

Valore di ritorno

- In caso di successo, `0`
- In caso di errore, `-1` , `errno` è settato

Dipartimento di Ingegneria informatica
automatica e gestionale Antonio Ruberti

Sapienza
Università di Roma

Funzioni Primitive TCP

Protocollo con messaggi testuali

Implementazione un protocollo client-server basato su messaggi di testo

- Il server è in ascolto su una porta nota
- Il client si connette al server
- Inizia uno scambio di messaggi di testo secondo uno schema predefinito («protocollo»)
- Protocollo di base
  - Il client invia una richiesta al server
  - Il server riceve la richiesta, la elabora, produce una risposta
  - Il server invia la risposta al client

Dipartimento di Ingegneria informatica
automatica e gestionale Antonio Ruberti

Sapienza
Università di Roma

EchoServer

Lab07, es1

Dipartimento di Ingegneria informatica
automatica e gestionale Antonio Ruberti



Sapienza
Università di Roma

Invio e Ricezione messaggi su Socket

Lab07 es1: EchoServer

Server single-thread in ascolto su una porta nota

Il client si connette al server:

1. Il server invia un welcome message
2. L'utente inserisce da terminale un messaggio
3. Il client invia il messaggio inserito al server
4. Se il messaggio inviato dal client è «QUIT», entrambi terminano la connessione.
5. In caso contrario, il server risponde con lo stesso messaggio ricevuto. Entrambi ripartono dal punto 2.

Sorgenti: `client.c` e `server.c`

Dipartimento di Ingegneria informatica
automatica e gestionale Antonio Ruberti

Sapienza
Università di Roma

Invio e Ricezione messaggi su Socket

Lab07 es1: EchoServer (Client)

Esercizio (lato client)

- Completare le parti mancanti, relative a:
  - Creazione e distruzione socket
  - Instaurare una connessione con il server
  - Invio/ricezione di messaggi via socket (gestire letture/scritture parziali)
    - Attenzione: non conosciamo la dimensione del messaggio
- Per l'esecuzione, lanciare `prof_server` e `client` su terminali diversi

Dipartimento di Ingegneria informatica
automatica e gestionale Antonio Ruberti

Sapienza
Università di Roma

Invio e Ricezione messaggi su Socket

Lab07 es1: EchoServer (Server)

Esercizio (lato server)

- Completare le parti mancanti, relative a:
  - Creazione, apertura e distruzione socket
  - Accettare una connessione in ingresso
  - Invio/ricezione di messaggi via socket (gestire letture/scritture parziali)
    - Attenzione: non conosciamo la dimensione del messaggio

- Per l'esecuzione, lanciare `server` e `prof_client` su terminali diversi

- Se tutto funziona, lanciare `server` e `client` su terminali diversi

Dipartimento di Ingegneria informatica
automatica e gestionale Antonio Ruberti

Sapienza
Università di Roma

UDP: Funzioni Primitive

Client/Server: Socket
Protocollo: messaggi

Dipartimento di Ingegneria informatica
automatica e gestionale Antonio Ruberti



Sapienza
Università di Roma

Funzioni Primitive

UDP: Client / Server

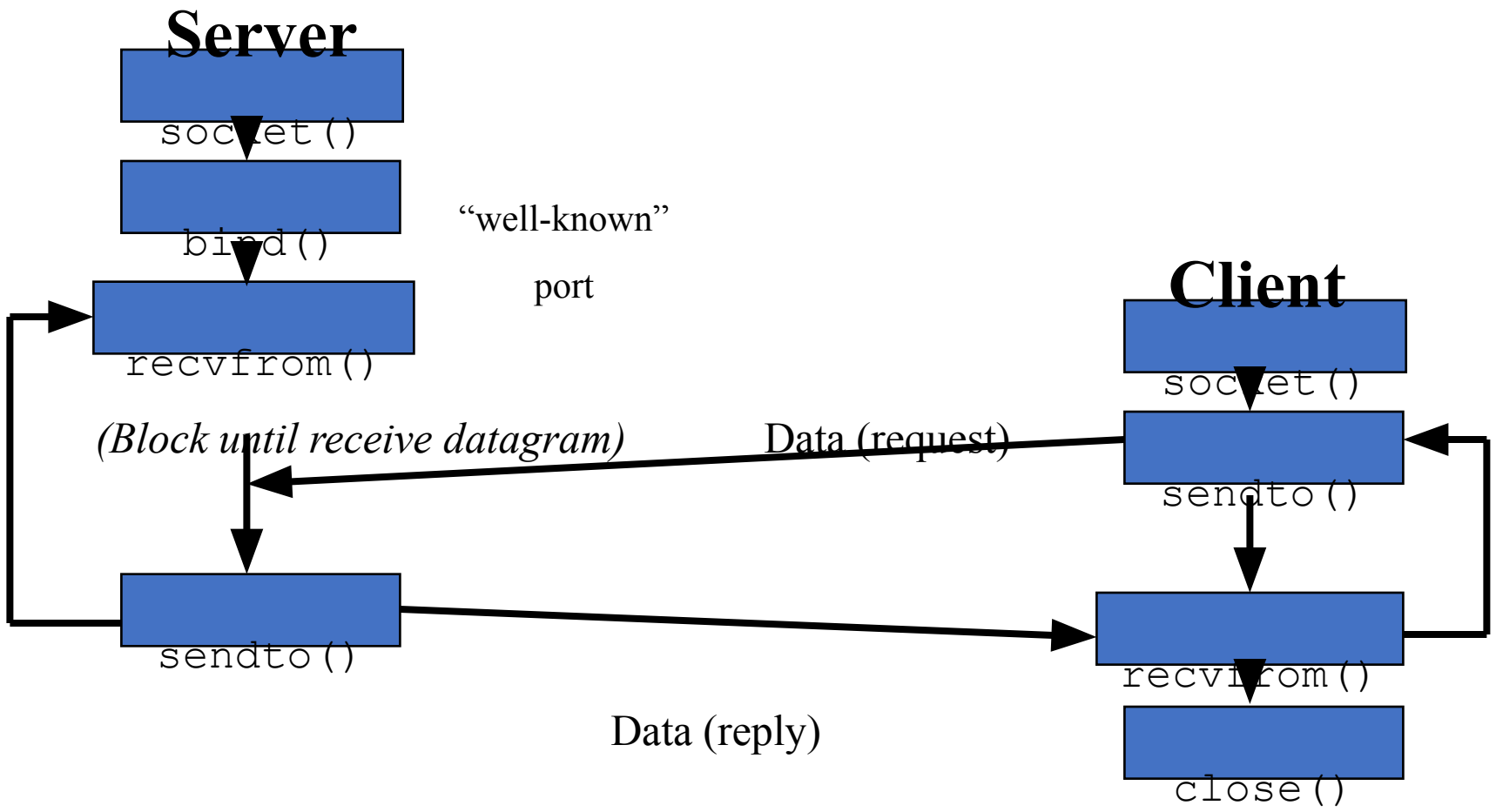
No “handshake”

`accept()-connect()`

(SYN – SYN/ACK – ACK)

- No simultaneous close

- No `fork()` for concurrent servers!



Funzioni Primitive UDP

Socket (Server, Client)

Richiedere al kernel di poter comunicare

- `socket()` creare i descrittori di comunicazione (struttura dati del kernel) per un **protocollo** supportato (`/etc/protocols`)

Mettersi in ascolto su una porta nota

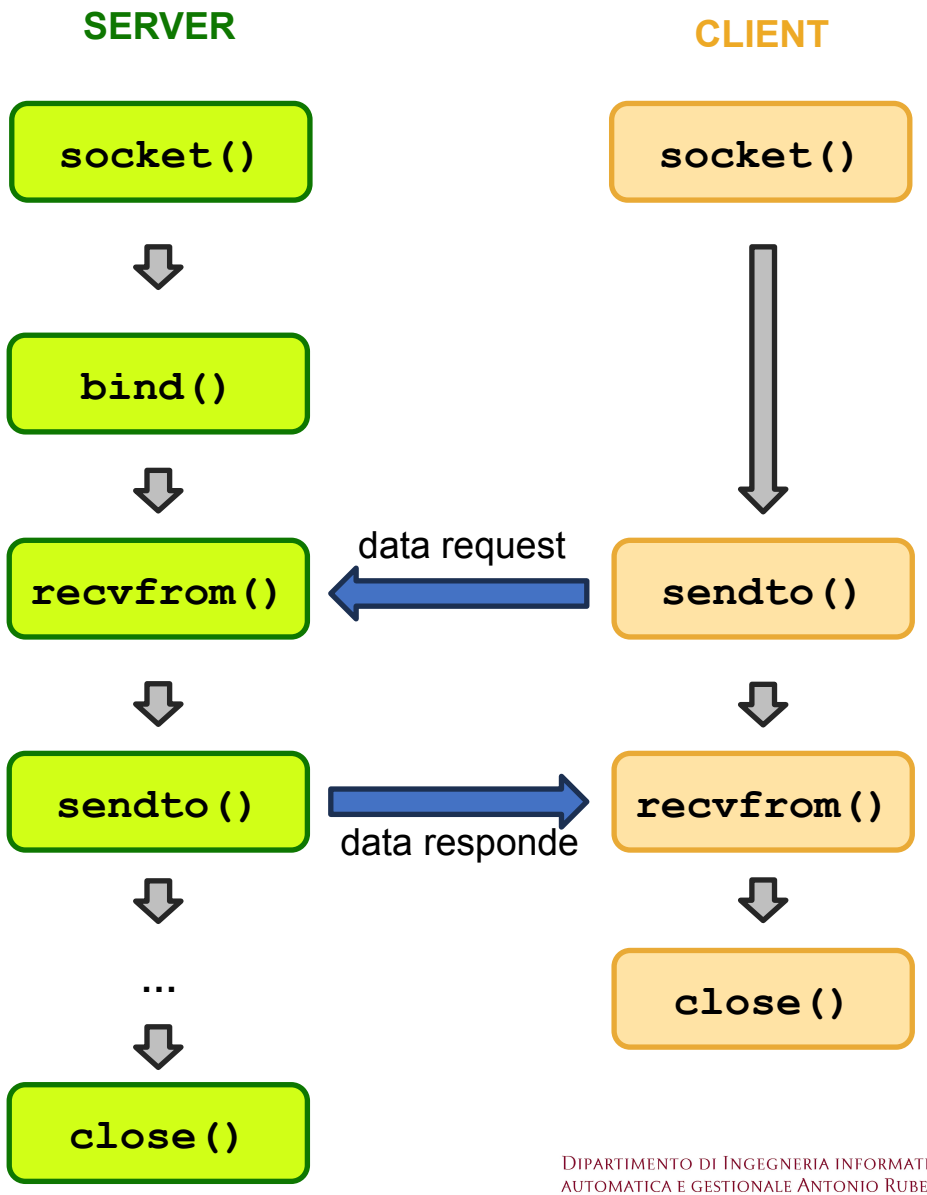
- `bind()` collegare socket ed indirizzo locale: al processo arriveranno tutte le comunicazioni sulla **porta dichiarata** (`/etc/services`)
- `listen()` dichiarare il **back-log** (numero max di connessioni accettabili)
- `accept()` avviare l'attesa delle richieste di comunicazione

Inviare una richiesta di comunicazione

- `connect()` inviare una richiesta di comunicazione ad un **indirizzo:porta**

Scambiarsi le informazioni tra server e client

- `sendto()` mandare messaggi sulla socket (indirizzo:porta)
- `recvfrom()` ricevere messaggi sulla socket (indirizzo:porta)
- `close()` liberare le strutture dati del kernel (una volta terminato lo scambio di messaggi)



Dipartimento di Ingegneria informatica automatica e gestionale Antonio Ruberti

Sapienza Università di Roma

Funzioni Primitive UDP

Socket (Server, Client): funzione `socket()`

```
int socket(int family, int type, int protocol);
```

Crea una socket, ossia un endpoint di comunicazione

Argomenti

- `family`: per i nostri scopi, `AF_INET`
  (vedi struttura dati `struct sockaddr_in`)
- `type`: per i nostri scopi, `SOCK_DGRAM` (protocollo UDP)
  - Precedentemente abbiamo usato `SICK_STREAM` (protocollo TCP)
- `protocol`: per i nostri scopi, `0`

Valore di ritorno

- In caso di successo, il descrittore della socket
- In caso di errore, `-1`, `errno` è settato

Dipartimento di Ingegneria informatica
automatica e gestionale Antonio Ruberti

Sapienza
Università di Roma

Funzioni Primitive UDP

Socket (invio messaggi): funzione `sendto()`

La funzione `sendto()` è definita in `sys/socket.h`

```
ssize_t sendto(int fd, const void *buf, size_t n, int flags,        const struct
sockaddr *dest_addr, socklen_t l );
```

- `fd, buf, n, flags`: come in `send()`
- `dest_addr, l`: come in `connect()`

Ritorna il numero di byte realmente scritti, o `-1` in caso di errore

- Default: semantica **bloccante**
  - Se buffer di invio nel kernel non contiene spazio sufficiente per il messaggio da inviare, rimane bloccata in attesa…

Funzioni Primitive UDP

Socket (ricezione messaggi): funzione `recvfrom()`

La funzione `recvfrom()` è definita in `sys/socket.h`

```
ssize_t recv(int fd, void *buf, size_t n, int flags,
struct sockaddr *src_addr, socklen_t *addrlen );
```

- `fd, buf, n, flags`: come in recv()
- Se `src_addr` è una variabile la funzione inserisce le informazione del mittente nella variabile e inserisce in `addrlen` la lunghezza della struttura
  - Utile per rispondere o distinguere tra più possibili mittenti
- Se `src_addr` è `NULL`, non salva il mittente, `addrlen` non viene modificato e può essere `NULL`
  - Utile quando non ci interessa sapere chi ha inviato il messaggio

Ritorna il numero di byte realmente letti, o `-1` in caso di errore

- Ritorna `0` in caso di connessione chiusa
- Default: semantica **bloccante**
  - Se l'altro endpoint non invia nulla, rimane bloccata in attesa
  - Nel client andrebbe modificata la socket con un timeout, ma tralasciamo questo aspetto
  - Trasferisce i dati disponibili fino a quel momento nel buffer del kernel, entro il limite di `n` bytes, piuttosto che restare in attesa di ricevere l'intera quantità specificata…

EchoServer UDP

Lab07, es2

Dipartimento di Ingegneria informatica
automatica e gestionale Antonio Ruberti



Sapienza
Università di Roma

Invio e Ricezione messaggi su Socket UDP

Lab07 es2: EchoServer

Server single-thread in ascolto su una porta nota

Modificare il codice dell'esercizio precedente per supportare una connessione UDP

Sorgenti: `client.c` e `server.c`

Dipartimento di Ingegneria informatica
automatica e gestionale Antonio Ruberti

Sapienza
Università di Roma

Autovalutazione

https://forms.gle/54rQUAdPf38M2yNm8